

E-Commerce Product Review and Rating Management System

1.1. REVIEW TABLE

Attribute	Data Type	Constraint
Review_ID	NUMBER	Primary Key
Customer_ID	NUMBER	—
Product_ID	NUMBER	—
Review_Text	VARCHAR2(500)	—
Review_Date	DATE	—

1.2. RATING TABLE

Attribute	Data Type	Constraint
Rating_ID	NUMBER	Primary Key
Review_ID	NUMBER	—
Product_ID	NUMBER	—
Rating	NUMBER(1)	CHECK (Rating BETWEEN 1 AND 5)

2. TABLE RELATIONSHIPS

Review_ID connects the Review and Rating records, while Product_ID groups reviews and ratings for each product.

3. TABLE CREATION

3.1. CREATE REVIEW TABLE

```
SQL> CREATE TABLE Review (  
2     Review_ID NUMBER PRIMARY KEY,  
3     Customer_ID NUMBER,  
4     Product_ID NUMBER,  
5     Review_Text VARCHAR2(500),  
6     Review_Date DATE  
7 );  
  
Table created.  
  
SQL> CREATE TABLE Rating (  
2     Rating_ID NUMBER PRIMARY KEY,  
3     Review_ID NUMBER,  
4     Product_ID NUMBER,  
5     Rating NUMBER(1) CHECK (Rating BETWEEN 1 AND 5)  
6 );  
  
Table created.  
  
SQL> INSERT INTO Review VALUES  
2 (1, 101, 201, 'Excellent bike and very smooth', SYSDATE);  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO Review VALUES  
2 (2, 102, 201, 'Good performance and quality', SYSDATE);  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO Review VALUES
```

3.2. CREATE RATING TABLE

```
SQL> CREATE TABLE Review (  
2     Review_ID NUMBER PRIMARY KEY,  
3     Customer_ID NUMBER,  
4     Product_ID NUMBER,  
5     Review_Text VARCHAR2(500),  
6     Review_Date DATE  
7 );  
  
Table created.  
  
SQL> CREATE TABLE Rating (  
2     Rating_ID NUMBER PRIMARY KEY,  
3     Review_ID NUMBER,  
4     Product_ID NUMBER,  
5     Rating NUMBER(1) CHECK (Rating BETWEEN 1 AND 5)  
6 );  
  
Table created.  
  
SQL> INSERT INTO Review VALUES  
2 (1, 101, 201, 'Excellent bike and very smooth', SYSDATE);  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO Review VALUES  
2 (2, 102, 201, 'Good performance and quality', SYSDATE);  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO Review VALUES
```

4. REVIEW DATA INSERTION

```
SQL>
SQL> INSERT INTO Review VALUES
  2 (5, 105, 203, 'Amazing quality and performance', SYSDATE);

1 row created.

SQL> INSERT INTO Rating VALUES (1, 1, 201, 5);

1 row created.

SQL> INSERT INTO Rating VALUES (2, 2, 201, 4);

1 row created.

SQL> INSERT INTO Rating VALUES (3, 3, 202, 5);

1 row created.

SQL> INSERT INTO Rating VALUES (4, 4, 202, 4);

1 row created.

SQL> INSERT INTO Rating VALUES (5, 5, 203, 5);

1 row created.

SQL> SELECT
  2     Review_ID,
  3     Customer_ID,
  4     Product_ID,
  5     Review_Text,
  6     Review_Date
```

```
REVIEW_ID CUSTOMER_ID PRODUCT_ID
-----
REVIEW_TEXT
-----
REVIEW_DA
-----
      1         101         201
Excellent bike and very smooth
06-OCT-26

      2         102         201
Good performance and quality
06-OCT-26

      REVIEW_ID CUSTOMER_ID PRODUCT_ID
      -----
      REVIEW_TEXT
      -----
      REVIEW_DA
      -----
      3         103         202
Very comfortable and stylish
06-OCT-26

      4         104         202
Good product but expensive

      REVIEW_ID CUSTOMER_ID PRODUCT_ID
      -----
      REVIEW_TEXT
      -----
      REVIEW_DA
```

5. RATING DATA INSERTION

```
SQL>
SQL> INSERT INTO Review VALUES
2  (5, 105, 203, 'Amazing quality and performance', SYSDATE);
1 row created.

SQL> INSERT INTO Rating VALUES (1, 1, 201, 5);
1 row created.

SQL> INSERT INTO Rating VALUES (2, 2, 201, 4);
1 row created.

SQL> INSERT INTO Rating VALUES (3, 3, 202, 5);
1 row created.

SQL> INSERT INTO Rating VALUES (4, 4, 202, 4);
1 row created.

SQL> INSERT INTO Rating VALUES (5, 5, 203, 5);
1 row created.

SQL> SELECT
2     Review_ID,
3     Customer_ID,
4     Product_ID,
5     Review_Text,
6     Review_Date
```

Rating_ID	Review_ID	Product_ID	Rating
1	1	201	5
2	2	201	4
3	3	202	5
4	4	202	4
5	5	203	5
6	6	203	4
7	7	204	5
8	8	204	4
9	9	205	5
10	10	205	4

6. RETRIEVE PRODUCT REVIEW DETAILS

```
SQL> INSERT INTO Review VALUES (6, 106, 203, 'Very satisfied with the product', SYSDATE);
1 row created.

SQL> INSERT INTO Review VALUES (7, 107, 204, 'Excellent features and design', SYSDATE);
1 row created.

SQL> INSERT INTO Review VALUES (8, 108, 204, 'Good bike for daily use', SYSDATE);
1 row created.

SQL> INSERT INTO Review VALUES (9, 109, 205, 'Great performance and mileage', SYSDATE);
1 row created.

SQL> INSERT INTO Review VALUES (10, 110, 205, 'Nice product and good service', SYSDATE);
1 row created.

SQL> SELECT * FROM Review;

REVIEW_ID CUSTOMER_ID PRODUCT_ID
-----
REVIEW_TEXT
-----
REVIEW_DA
-----
          1          101          201
Excellent bike and very smooth
06-OCT-26

          2          102          201
Good performance and quality
```

```
          5          108          203
Amazing quality and performance
06-OCT-26

SQL> SELECT
2   Product_ID,
3   AVG(Rating) AS Average_Rating
4 FROM Rating
5 GROUP BY Product_ID;

PRODUCT_ID AVERAGE_RATING
-----
          201             4.5
          202             4.5
          203              5

SQL> SELECT
2   Product_ID,
3   AVG(Rating) AS Average_Rating
4 FROM Rating
5 GROUP BY Product_ID
6 HAVING AVG(Rating) >= 4;

PRODUCT_ID AVERAGE_RATING
-----
          201             4.5
          202             4.5
          203              5

SQL> INSERT INTO Review VALUES (1, 101, 201, 'Excellent bike and smooth performance', SYSDATE);
INSERT INTO Review VALUES (1, 101, 201, 'Excellent bike and smooth performance', SYSDATE)
```

7. CALCULATE AVERAGE PRODUCT RATINGS

10 rows selected.

```
SQL> SELECT
  2     Product_ID,
  3     AVG(Rating) AS Average_Rating
  4 FROM Rating
  5 GROUP BY Product_ID;
```

PRODUCT_ID	AVERAGE_RATING
201	4.5
202	4.5
203	4.5
204	4.5
205	4.5

```
SQL> SELECT
  2     Product_ID,
  3     AVG(Rating) AS Average_Rating
  4 FROM Rating
  5 GROUP BY Product_ID
  6 HAVING AVG(Rating) >= 4;
```

PRODUCT_ID	AVERAGE_RATING
201	4.5
202	4.5
203	4.5
204	4.5
205	4.5

Product_ID	Average_Rating
201	4.5
202	4.5
203	4.5
204	4.5
205	4.5

8. IDENTIFY HIGHLY RATED PRODUCTS

```
10 rows selected.

SQL> SELECT
  2     Product_ID,
  3     AVG(Rating) AS Average_Rating
  4 FROM Rating
  5 GROUP BY Product_ID;

PRODUCT_ID AVERAGE_RATING
-----
201         4.5
202         4.5
203         4.5
204         4.5
205         4.5

SQL> SELECT
  2     Product_ID,
  3     AVG(Rating) AS Average_Rating
  4 FROM Rating
  5 GROUP BY Product_ID
  6 HAVING AVG(Rating) >= 4;

PRODUCT_ID AVERAGE_RATING
-----
201         4.5
202         4.5
203         4.5
204         4.5
205         4.5
```

Using HAVING AVG(Rating) >= 4, products 201, 202, 203, 204 and 205 are identified as highly rated.

9. PRODUCT RATING REPORTS

Product_ID	Average_Rating	Status
201	4.5	Highly Rated
202	4.5	Highly Rated
203	4.5	Highly Rated
204	4.5	Highly Rated
205	4.5	Highly Rated

Result: The Review and Rating tables were created successfully. Customer feedback and ratings were stored, product review details were retrieved, average product ratings were calculated using aggregate functions, and highly rated products were identified.